FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA – FIAP
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

GUSTAVO FERREIRA SCHMIDT

CIDADES ESG INTELIGENTES:

containerização e pipeline CI/CD da API de Gestão de Resíduos e Reciclagem

SÃO PAULO
2026

GUSTAVO FERREIRA SCHMIDT

CIDADES ESG INTELIGENTES:

containerização e pipeline CI/CD da API de Gestão de Resíduos e Reciclagem

Documentação técnica apresentada à Faculdade de Informática e Administração Paulista – FIAP, curso de Tecnologia em Análise e Desenvolvimento de Sistemas, como requisito de avaliação da atividade da fase “Navegando pelo mundo DevOps”.

RM: 562688

SÃO PAULO

2026

SUMÁRIO

1 INTRODUÇÃO.....3
2 ARQUITETURA DA SOLUÇÃO.....4
3 PIPELINE CI/CD.....5
3.1 Ferramenta e gatilhos.....5
3.2 Etapas do pipeline.....5
3.3 Configuração de variáveis e segredos.....6
3.4 Lógica e boas práticas.....6
4 CONTAINERIZAÇÃO E ORQUESTRAÇÃO.....8
4.1 Dockerfile.....8
4.2 Docker Compose.....9
4.3 Comandos utilizados.....9
4.4 Imagem criada.....9
5 EVIDÊNCIAS DE FUNCIONAMENTO.....11
5.1 Execução do pipeline.....11
5.2 Ambientes local, staging e produção.....13
6 DESAFIOS ENCONTRADOS E SOLUÇÕES.....15
7 CONSIDERAÇÕES FINAIS.....16
REFERÊNCIAS.....17
APÊNDICE A – CHECKLIST DE ENTREGA.....18

1 INTRODUÇÃO

Este trabalho apresenta a aplicação de práticas de DevOps sobre a API RESTful de Gestão de Resíduos e Reciclagem, desenvolvida em C# com .NET 8 nas fases anteriores do curso, dentro do tema ESG (*Environmental, Social and Governance*). A API registra descartes de resíduos em pontos de coleta e aplica automaticamente regras de compliance ambiental, como a atualização da ocupação dos pontos, a geração de alertas de capacidade, o bloqueio de pontos cheios e a consolidação de relatórios mensais, com autenticação por JWT.

O objetivo da atividade foi adaptar essa aplicação para simular um ambiente de produção real, automatizando todo o seu ciclo de vida. Para isso, foram definidos os seguintes objetivos específicos:

a) containerizar a aplicação com Docker;
b) orquestrar a aplicação e o banco de dados com Docker Compose, utilizando volumes, redes e variáveis de ambiente;
c) implementar um pipeline de integração e entrega contínuas (CI/CD) com etapas automatizadas de build, testes e deploy;
d) realizar o deploy automatizado em dois ambientes distintos: staging e produção.

O código-fonte está disponível no repositório https://github.com/Guschmidt19/cidades-esg-inteligentes. Os ambientes publicados podem ser acessados em https://cidades-esg-staging-zlcv.onrender.com/swagger (staging) e https://cidades-esg-producao.onrender.com/swagger (produção).

2 ARQUITETURA DA SOLUÇÃO

A solução foi organizada em camadas, de modo que cada ferramenta tenha uma responsabilidade bem definida no ciclo de vida da aplicação. O Quadro 1 resume os componentes adotados.

Quadro 1 – Componentes da solução

Camada	Tecnologia	Função
Código-fonte	GitHub (branch `main`)	Versionamento; cada *push* dispara o pipeline
Integração e entrega contínuas	GitHub Actions	Build, testes, construção da imagem e deploy
Registro de imagens	GitHub Container Registry (GHCR)	Armazenamento das imagens versionadas pelo commit
Execução	Render (dois serviços Docker)	Ambientes de staging e de produção, com URLs públicas
Banco de dados	PostgreSQL – Neon (nuvem) e container (local)	Bancos isolados `esg_staging` e `esg_production`

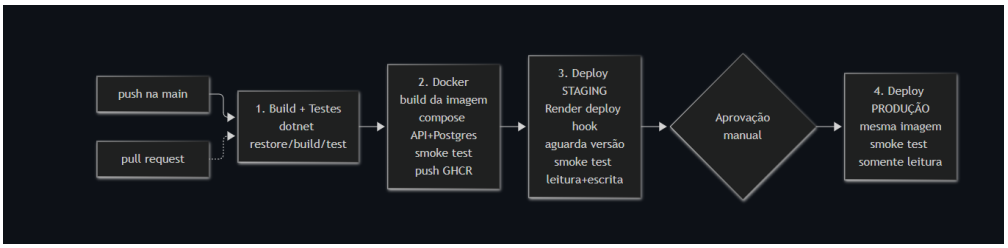
Fonte: Elaborado pelo autor (2026).

Optou-se pelo PostgreSQL como banco padrão porque o Oracle utilizado no projeto original não é adequado para execução em containers nem acessível a partir dos serviços em nuvem. O provedor de banco tornou-se configurável por variável de ambiente (`Database__Provider`), de modo que o suporte ao Oracle foi preservado.

3 PIPELINE CI/CD

O pipeline foi implementado com o GitHub Actions e está definido no arquivo `.github/workflows/ci-cd.yml`. A Figura 1 apresenta o fluxo completo, desde o envio do código até o deploy em produção.

Figura 1 – Fluxo do pipeline CI/CD



Fonte: Elaborado pelo autor (2026).

3.1 Ferramenta e gatilhos

O GitHub Actions foi escolhido por estar integrado ao repositório, dispensar infraestrutura própria e oferecer recursos de ambientes (*environments*) com segredos e regras de aprovação. O pipeline é executado nas seguintes situações:

a) *push* na branch `main`: executa todas as etapas, inclusive os deploys;
b) *pull request* para a `main`: executa apenas build, testes e teste do container, sem publicar nada;
c) execução manual (`workflow_dispatch`), disponível na interface do GitHub.

3.2 Etapas do pipeline

O pipeline é composto por quatro jobs sequenciais, descritos no Quadro 2. Cada job só é executado se o anterior for concluído com sucesso.

Quadro 2 – Etapas do pipeline

Job	Descrição	Condição de falha
1. Build e testes	Executa `dotnet restore`, `dotnet build` e `dotnet test`, com 12 testes automatizados em xUnit sobre banco InMemory. Gera resumo e artefato com os resultados.	Erro de compilação ou teste reprovado
2. Docker	Constrói a imagem com a tag `sha-<commit>`, sobe API e PostgreSQL com Docker Compose no próprio runner, executa o *smoke test* e publica a imagem no GHCR.	Container não inicia, banco não conecta ou endpoint responde incorretamente
3. Deploy staging	Aciona o *deploy hook* do Render com a imagem exata, aguarda a nova versão em `/api/info` e executa o *smoke test* com leitura e escrita.	Deploy não concluído em 15 minutos ou *smoke test* reprovado

Job	Descrição	Condição de falha
4. Deploy produção	Após aprovação manual, publica a mesma imagem validada em staging e executa o *smoke test* somente leitura.	Idem ao job 3

Fonte: Elaborado pelo autor (2026).

3.3 Configuração de variáveis e segredos

O Quadro 3 apresenta as configurações cadastradas no GitHub para que o pipeline acesse os ambientes.

Quadro 3 – Variáveis, segredos e regras configurados no GitHub

Tipo	Nome	Finalidade
Variável do repositório	`STAGING_URL`	Endereço de staging; habilita o job 3
Variável do repositório	`PRODUCTION_URL`	Endereço de produção; habilita o job 4
Segredo do environment *staging*	`RENDER_DEPLOY_HOOK_URL`	Aciona o deploy no serviço de staging
Segredo do environment *production*	`RENDER_DEPLOY_HOOK_URL`	Aciona o deploy no serviço de produção
Regra do environment *production*	Required reviewers	Exige aprovação manual antes do deploy

Fonte: Elaborado pelo autor (2026).

3.4 Lógica e boas práticas

A lógica do pipeline segue o princípio *build once, deploy many*: a imagem é construída e testada uma única vez, e o mesmo artefato é promovido para staging e, depois, para produção. Entre os ambientes muda apenas a configuração, fornecida por variáveis de ambiente. Além disso, foram adotadas as seguintes práticas:

a) **rastreabilidade**: a tag da imagem corresponde ao SHA do commit, e a API expõe essa versão no endpoint `/api/info`, o que permite ao pipeline confirmar que o deploy foi efetivamente concluído;
b) **portões de qualidade**: a produção depende do sucesso em staging e exige aprovação manual de um revisor (*required reviewers*);
c) **testes pós-deploy**: um *smoke test* verifica a saúde da API e do banco, os endpoints de negócio, o bloqueio de requisições sem token (HTTP 401) e, em staging, o fluxo autenticado de registro de descarte (HTTP 201);
d) **segredos fora do código**: os *deploy hooks* ficam em segredos dos *environments* do GitHub, e as credenciais do banco e as chaves JWT ficam nas variáveis de cada serviço no Render;

e) **controle de concorrência**: o bloco `concurrency` impede dois deploys simultâneos na mesma branch.

4 CONTAINERIZAÇÃO E ORQUESTRAÇÃO

A aplicação foi containerizada com Docker, garantindo que o mesmo artefato seja executado de forma idêntica no computador do desenvolvedor, no pipeline e nos ambientes em nuvem.

4.1 Dockerfile

O Dockerfile adota a estratégia *multi-stage build*, dividida em quatro estágios: *build* (restauração de dependências e compilação com o SDK do .NET 8), *test* (execução opcional dos testes), *publish* (geração dos binários) e *final* (imagem de execução contendo apenas o runtime ASP.NET). A Figura 2 apresenta a estrutura resumida do arquivo.

Figura 2 – Estrutura resumida do Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
COPY *.sln *.csproj  ->  RUN dotnet restore          # camada em cache
COPY src/ tests/     ->  RUN dotnet build -c Release
FROM build AS test   ->  RUN dotnet test             # opcional
FROM build AS publish -> RUN dotnet publish -o /app/publish
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
ARG APP_VERSION=local                # SHA do commit (pipeline)
COPY --from=publish --chown=app:app /app/publish .
ENV ASPNETCORE_HTTP_PORTS=8080 ASPNETCORE_ENVIRONMENT=Production
USER app                             # usuário sem privilégios
HEALTHCHECK CMD curl -fsS http://localhost:${PORT:-8080}/health/live
ENTRYPOINT ["dotnet", "ESG.GestaoResiduos.API.dll"]
```

Fonte: Elaborado pelo autor (2026).

As principais decisões adotadas na construção da imagem foram:

a) **imagem final reduzida**: o SDK existe apenas nos estágios de build, e a imagem publicada contém somente o runtime e a aplicação;
b) **aproveitamento de cache**: os arquivos de projeto são restaurados antes do código-fonte, evitando novo download de dependências a cada alteração;
c) **segurança**: a aplicação é executada com o usuário sem privilégios `app`, e o `.dockerignore` impede a inclusão de arquivos sensíveis, como o `.env`;
d) **portabilidade**: a porta é lida da variável `PORT` (Render) ou `ASPNETCORE_HTTP_PORTS` (local), permitindo que a mesma imagem seja executada em qualquer ambiente;
e) **resiliência**: o container possui *healthcheck*, e a API realiza até dez tentativas de conexão com o banco durante a inicialização.

4.2 Docker Compose

A orquestração local foi feita com Docker Compose, conforme o Quadro 4. A API só é iniciada após o banco ser considerado saudável, e o banco não publica porta para o computador hospedeiro, sendo acessado apenas pela rede interna, pelo nome do serviço.

Quadro 4 – Serviços definidos no docker-compose.yml

Serviço	Imagem	Características
`db`	`postgres:16-alpine`	*Healthcheck* com `pg_isready`; volume `cidades-esg-pgdata`; rede `backend`; sem porta publicada
`api`	Construída pelo Dockerfile	Inicia após o banco saudável (`service_healthy`); porta 8080; redes `backend` e `frontend`
`adminer`	`adminer:4.8.1`	Opcional (perfil `tools`); interface web de consulta ao banco

Fonte: Elaborado pelo autor (2026).

O volume nomeado garante a persistência dos dados entre reinicializações. As variáveis de ambiente são lidas de um arquivo `.env`, criado a partir do modelo `.env.example` e mantido fora do controle de versão. Dentro do container, o ASP.NET Core lê variáveis no formato `Secao__Chave` (por exemplo, `ConnectionStrings__DefaultConnection`), que sobrescrevem os valores do `appsettings.json`.

4.3 Comandos utilizados

Quadro 5 – Comandos Docker utilizados no projeto

Comando	Finalidade
`docker compose up -d --build`	Constrói a imagem e inicia API e banco em segundo plano
`docker compose ps`	Exibe os serviços e o estado do *healthcheck*
`docker compose logs -f api`	Acompanha os logs da API
`docker compose down -v`	Encerra os serviços e remove o volume do banco
`docker build --target test .`	Executa os testes dentro do Docker
`bash scripts/smoke-test.sh <url> --escrita`	Verifica a API em execução de ponta a ponta

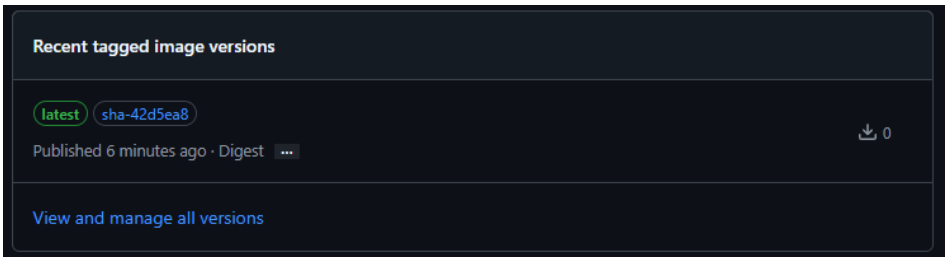
Fonte: Elaborado pelo autor (2026).

4.4 Imagem criada

A imagem é publicada automaticamente pelo pipeline no GitHub Container Registry, no endereço `ghcr.io/guschmidt19/cidades-esg-inteligentes`. Cada

execução gera duas tags: `sha-<commit>`, imutável e utilizada nos deploys, e `latest`, conforme a Figura 3.

Figura 3 – Imagem publicada no GitHub Container Registry



Fonte: Captura de tela do autor (2026).

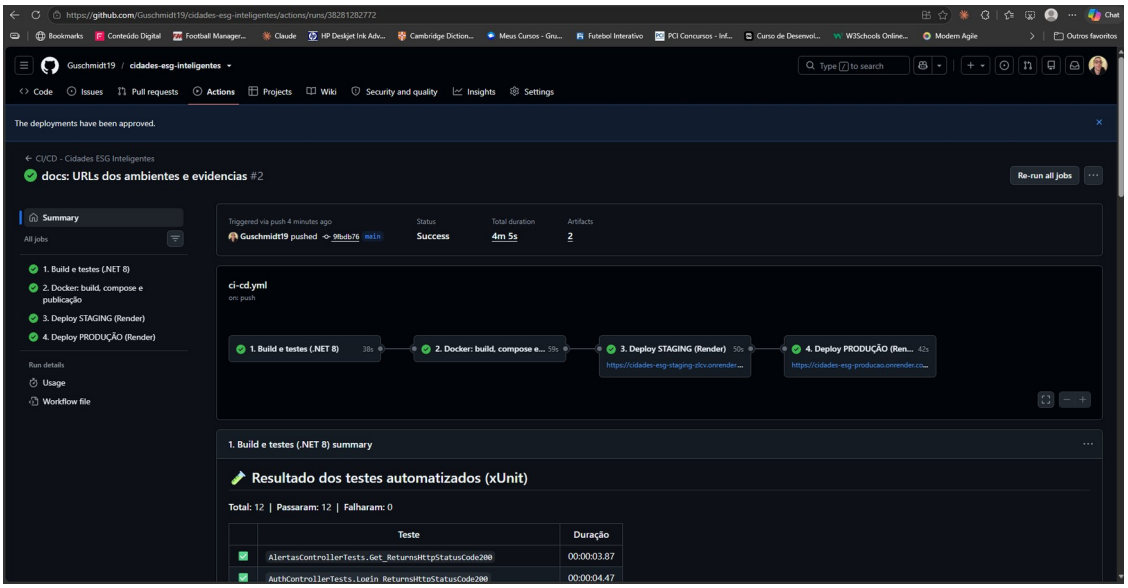
5 EVIDÊNCIAS DE FUNCIONAMENTO

Esta seção reúne as evidências de execução do pipeline e de funcionamento dos ambientes. As capturas referentes ao deploy correspondem ao commit `9fbdb76`.

5.1 Execução do pipeline

A Figura 4 mostra a execução completa do pipeline, com os quatro jobs concluídos com sucesso em 4 minutos e 5 segundos e os links dos ambientes publicados.

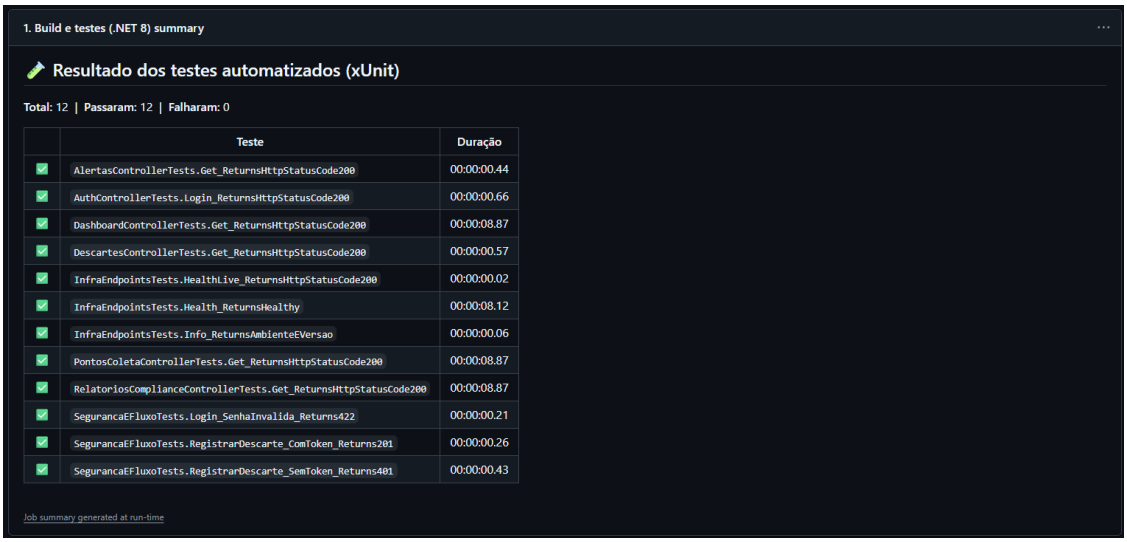
Figura 4 – Execução completa do pipeline no GitHub Actions



Fonte: Captura de tela do autor (2026).

A Figura 5 apresenta o resumo dos testes automatizados. Além da verificação do status HTTP 200 em cada controller, os testes cobrem os endpoints de infraestrutura e a segurança da API: bloqueio sem token (401), rejeição de senha inválida (422) e registro autenticado de descarte (201).

Figura 5 – Resultado dos testes automatizados (job 1)



1. Build e testes (.NET 8) summary

Resultado dos testes automatizados (xUnit)

Total: 12 | Passaram: 12 | Falharam: 0

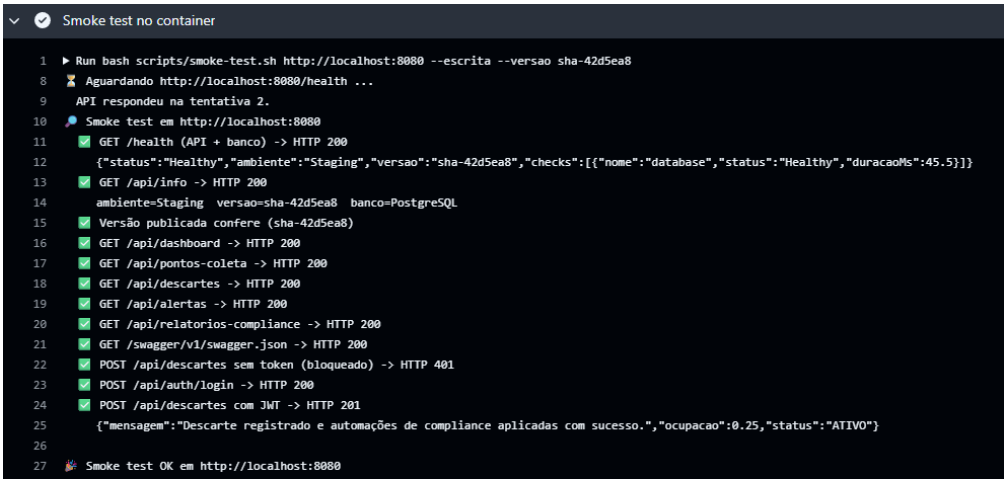
	Teste	Duração
✅	AlertasControllerTests.Get_ReturnsHttpStatusCode200	00:00:00.44
✅	AuthControllerTests.Login_ReturnsHttpStatusCode200	00:00:00.66
✅	DashboardControllerTests.Get_ReturnsHttpStatusCode200	00:00:08.87
✅	DescartesControllerTests.Get_ReturnsHttpStatusCode200	00:00:00.57
✅	InfraEndpointsTests.HealthLive_ReturnsHttpStatusCode200	00:00:00.02
✅	InfraEndpointsTests.Health_ReturnsHealthy	00:00:08.12
✅	InfraEndpointsTests.Info_ReturnsAmbienteEVersao	00:00:00.06
✅	PontosColetaControllerTests.Get_ReturnsHttpStatusCode200	00:00:08.87
✅	RelatoriosComplianceControllerTests.Get_ReturnsHttpStatusCode200	00:00:08.87
✅	SegurancaEFluxoTests.Login_SenhaInvalida_Returns422	00:00:00.21
✅	SegurancaEFluxoTests.RegistrarDescarte_ComToken_Returns201	00:00:00.26
✅	SegurancaEFluxoTests.RegistrarDescarte_SemToken_Returns401	00:00:00.43

Job summary generated at run-time

Fonte: Captura de tela do autor (2026).

A Figura 6 mostra o *smoke test* executado contra o container, com API e PostgreSQL orquestrados pelo Docker Compose dentro do próprio pipeline.

Figura 6 – Smoke test do container no pipeline (job 2)



Smoke test no container

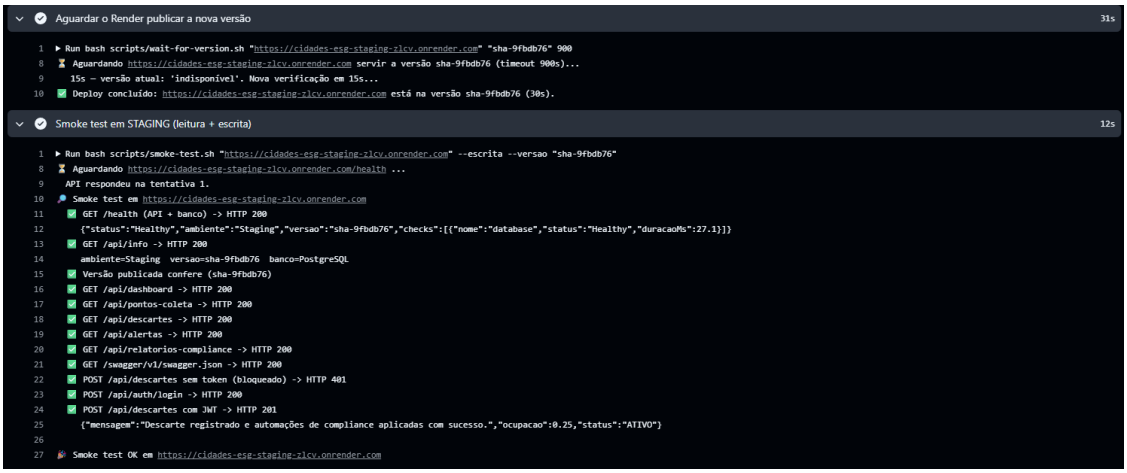
```
▶ Run bash scripts/smoke-test.sh http://localhost:8080 --escrita --versao sha-42d5ea8
⌛ Aguardando http://localhost:8080/health ...
  API respondeu na tentativa 2.
🔎 Smoke test em http://localhost:8080
  ✅ GET /health (API + banco) -> HTTP 200
     {"status":"Healthy","ambiente":"Staging","versao":"sha-42d5ea8","checks":[{"nome":"database","status":"Healthy","duracaoMs":45.5}]}
  ✅ GET /api/info -> HTTP 200
     ambiente=Staging  versao=sha-42d5ea8  banco=PostgreSQL
  ✅ Versão publicada confere (sha-42d5ea8)
  ✅ GET /api/dashboard -> HTTP 200
  ✅ GET /api/pontos-coleta -> HTTP 200
  ✅ GET /api/descartes -> HTTP 200
  ✅ GET /api/alertas -> HTTP 200
  ✅ GET /api/relatorios-compliance -> HTTP 200
  ✅ GET /swagger/v1/swagger.json -> HTTP 200
  ✅ POST /api/descartes sem token (bloqueado) -> HTTP 401
  ✅ POST /api/auth/login -> HTTP 200
  ✅ POST /api/descartes com JWT -> HTTP 201
     {"mensagem":"Descarte registrado e automações de compliance aplicadas com sucesso.","ocupacao":0.25,"status":"ATIVO"}

🎉 Smoke test OK em http://localhost:8080
```

Fonte: Captura de tela do autor (2026).

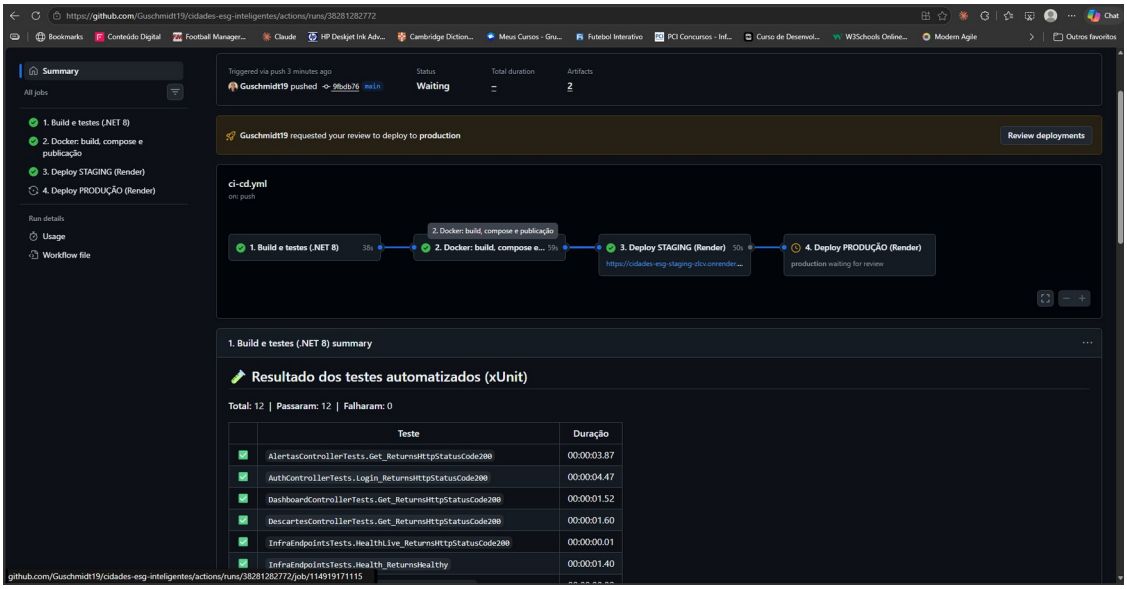
A Figura 7 apresenta o deploy em staging: o pipeline aguarda o Render publicar a nova versão e, em seguida, executa o *smoke test*. A Figura 8 mostra o portão de qualidade, no qual o pipeline aguarda aprovação manual antes do deploy em produção.

Figura 7 – Verificação de versão e smoke test em staging (job 3)



Fonte: Captura de tela do autor (2026).

Figura 8 – Aprovação manual exigida antes do deploy em produção (job 4)

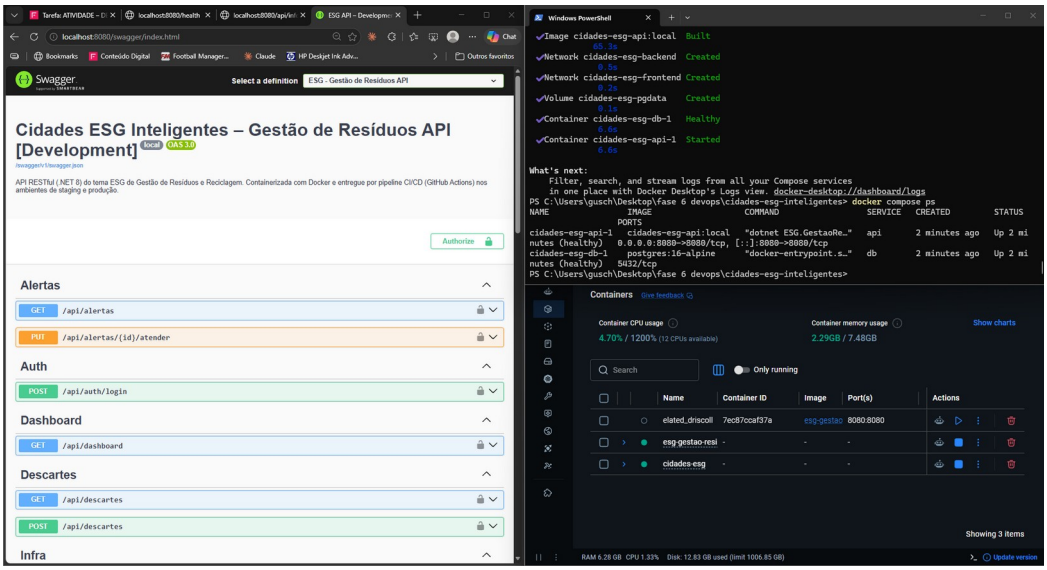


Fonte: Captura de tela do autor (2026).

5.2 Ambientes local, staging e produção

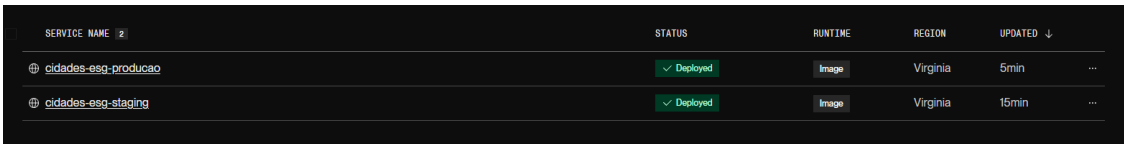
A Figura 9 mostra a aplicação em execução local, com os serviços `api` e `db` em estado saudável e a documentação Swagger acessível. A Figura 10 apresenta os dois serviços publicados no Render.

Figura 9 – Execução local com Docker Compose



Fonte: Captura de tela do autor (2026).

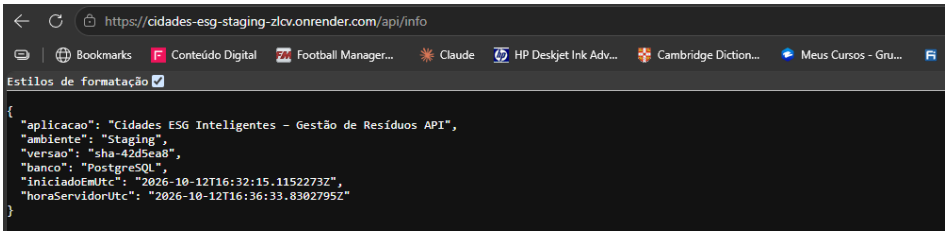
Figura 10 – Serviços de staging e produção publicados no Render



Fonte: Captura de tela do autor (2026).

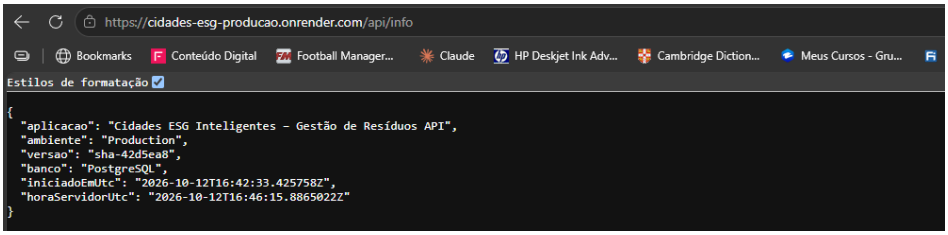
As Figuras 11 e 12 mostram o endpoint `/api/info` de cada ambiente, que confirma o nome do ambiente (Staging ou Production), o banco em uso e a versão da imagem. O ambiente de staging (https://cidades-esg-staging-zlcv.onrender.com) utiliza o banco `esg_staging`, e o de produção (https://cidades-esg-producao.onrender.com) utiliza o banco `esg_production`.

Figura 11 – Endpoint /api/info no ambiente de staging



Fonte: Captura de tela do autor (2026).

Figura 12 – Endpoint /api/info no ambiente de produção



Fonte: Captura de tela do autor (2026).

6 DESAFIOS ENCONTRADOS E SOLUÇÕES

Durante a adaptação do projeto surgiram desafios técnicos, vários deles observados diretamente nos logs, como a resposta da API apenas na segunda tentativa durante o teste do container, a porta 10000 atribuída pelo Render e os jobs de deploy ignorados na primeira execução do pipeline. O Quadro 6 relaciona os desafios e as soluções adotadas.

Quadro 6 – Desafios encontrados e soluções adotadas

Nº	Desafio	Solução adotada
1	Banco Oracle do projeto original inadequado para containers e inacessível a partir da nuvem	Provedor configurável por variável, com PostgreSQL no Docker e na nuvem e Oracle mantido como opção
2	Migrations específicas do Oracle (tipos como `NUMBER(10)`)	Criação do schema a partir do modelo no PostgreSQL (`EnsureCreated`) e migrations mantidas no Oracle
3	Banco demora a aceitar conexões na inicialização	`depends_on` com `service_healthy` e até dez tentativas de conexão no startup da API
4	Render define a porta pela variável `PORT` (10000)	Leitura de `PORT` pela API, com 8080 como padrão
5	Execução dos testes no CI sem banco real	`WebApplicationFactory` com EF Core InMemory e banco isolado por classe de teste
6	Quebras de linha CRLF do Windows impedem a execução de scripts no Linux	`.gitattributes` forçando LF e execução por `bash script.sh`
7	Confirmar a conclusão do deploy e a versão em execução	Tag da imagem igual ao SHA do commit, exposta em `/api/info` e verificada pelo pipeline
8	Render exige imagem existente, que só é criada pelo pipeline	Jobs de deploy condicionados às variáveis de URL, ignorados na primeira execução
9	Proteção de senhas e chaves em repositório público	Segredos no Render e nos *environments* do GitHub; chave JWT distinta por ambiente; `.env` fora do Git
10	Risco de uma versão com falha chegar diretamente à produção	Dependência do sucesso em staging e aprovação manual obrigatória

Fonte: Elaborado pelo autor (2026).

7 CONSIDERAÇÕES FINAIS

Os objetivos propostos foram atingidos. A aplicação ESG de Gestão de Resíduos foi containerizada e orquestrada com seu banco de dados, utilizando volumes, redes e variáveis de ambiente, e passou a ser entregue por um pipeline de CI/CD que compila, testa e publica automaticamente a mesma imagem nos ambientes de staging e de produção.

A verificação automática após cada deploy, aliada à aprovação manual antes da produção, aproxima o projeto das práticas adotadas em ambientes profissionais. Com isso, qualquer alteração enviada à branch principal percorre todo o fluxo de validação e publicação sem intervenção manual, exceto a aprovação final.

Como evoluções futuras, sugerem-se a adoção de migrations específicas para o PostgreSQL, a coleta centralizada de logs e métricas e a migração da orquestração para Kubernetes.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

DOCKER INC. **Docker Docs**. Disponível em: https://docs.docker.com. Acesso em: 12 out. 2026.

GITHUB. **GitHub Actions documentation**. Disponível em: https://docs.github.com/actions. Acesso em: 12 out. 2026.

MICROSOFT. **ASP.NET Core documentation**. Disponível em: https://learn.microsoft.com/aspnet/core. Acesso em: 12 out. 2026.

NEON. **Neon documentation**. Disponível em: https://neon.tech/docs. Acesso em: 12 out. 2026.

RENDER. **Render documentation**. Disponível em: https://render.com/docs. Acesso em: 12 out. 2026.

APÊNDICE A – CHECKLIST DE ENTREGA

Quadro 7 – Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	Sim
Dockerfile funcional	Sim
docker-compose.yml ou arquivos Kubernetes	Sim
Pipeline com etapas de build, teste e deploy	Sim
README.md com instruções e prints	Sim
Documentação técnica com evidências (PDF ou PPT)	Sim
Deploy realizado nos ambientes staging e produção	Sim

Fonte: Elaborado pelo autor (2026).